

POLITECNICO
MILANO 1863

Ingegneria del Software

Ingegneria Informatica – Prof. San Pietro

Esercitazione:

JML: specifica procedure

My Contacts:
Giuseppe Sorrentino
giuseppe.sorrentino@polimi.it

Specifica Procedure:

- Se vogliamo descrivere un metodo, ciò che ci interessa sapere è:
  1. Come devono essere i parametri in ingresso, ovvero quali sono le **pre-condizioni** del metodo.

Specifica Procedure:

- Se vogliamo descrivere un metodo, ciò che ci interessa sapere è:
  1. Come devono essere i parametri in ingresso, ovvero quali sono le **pre-condizioni** del metodo.
  2. Come saranno i parametri in uscita, ovvero quali saranno le **post-condizioni** del metodo.

Specifica Procedure:

- Se vogliamo descrivere un metodo, ciò che ci interessa sapere è:
  1. Come devono essere i parametri in ingresso, ovvero quali sono le **pre-condizioni** del metodo.
  2. Come saranno i parametri in uscita, ovvero quali saranno le **post-condizioni** del metodo.
  3. Cosa accade, quando ci sono situazioni impreviste.

Specifica Procedure:

- Se vogliamo descrivere un metodo, ciò che ci interessa sapere è:
  1. Come devono essere i parametri in ingresso, ovvero quali sono le **pre-condizioni** del metodo.
  2. Come saranno i parametri in uscita, ovvero quali saranno le **post-condizioni** del metodo.
  3. Cosa accade, quando ci sono situazioni impreviste.
  4. Quali variabili devo considerare modificate, in seguito all'uso del metodo.

Specifica Procedure: Requires

- **\\@Requires** consente di specificare le **pre-condizioni di un metodo.**
- Mi assicura che, se il parametro rispetta i requires, allora **so esattamente** come sarà dopo l'esecuzione del metodo: **non mi dice nulla** riguardo ai parametri che **non** rispettano i requisiti.

Specifica Procedure: Ensures

- **\\@Ensures** consente di specificare le **post-condizioni di un metodo.**
- Mi assicura che, se il parametro rispetta i requires, allora l'output sarà di un certo tipo.

Specifica Procedure: Signal

- **\\@Signal** mi dice con quali accezioni vengono gestite le **situazioni esterne al dominio dei requires.**
- In altre parole, il Requires mi dice come deve essere un parametro affinchè l'ensures possa essere applicata. Il Signal, che è sempre **mutuamente esclusivo con il requires e l'ensure**, mi dice cosa accade quando la condizione di requires ed ensures non è rispettata.
- Il Signal prevede il lancio di un'eccezione per gestire la situazione inattesa.
- In un signal, devo descrivere **tutto lo stato** a seguito del lancio dell'eccezione. Ovvero, devo specificare anche **ciò che non è cambiato**

Specifica Procedure: Assignable

- **\\@Assignable** mi consente di specificare quali variabili verrano modificate dal metodo

Procedure Totali e Parziali

- Una procedura si dice **totale** se sono esattamente in grado di definire come sarà lo stato, in qualsiasi valore del dominio dell'input.

Procedure Totali e Parziali

- Una procedura si dice **totale** se sono esattamente in grado di definire come sarà lo stato, in qualsiasi valore del dominio dell'input.
- In altre parole, le condizioni di Requires ed Ensures, assieme a quelle «previste» dalla signal, devono coprire TUTTO il dominio di input

Procedure Totali e Parziali

- Una procedura si dice **totale** se sono esattamente in grado di definire come sarà lo stato, in qualsiasi valore del dominio dell'input.
- In altre parole, le condizioni di Requires ed Ensures, assieme a quelle «previste» dalla signal, devono coprire TUTTO il dominio di input
- In caso contrario, la procedura si dice **parziale.**

Keyword e funzioni utili : \forall

- **\forall** mi consente di iterare lungo una serie di valori, come un «for»
- **La struttura** sarà:

  \forall (Valore, Dominio su cui itero, Condizione/Azione)

Keyword e funzioni utili : \exist

- **\forall** mi consente di iterare lungo una serie di valori, come un «for»
- **La struttura** sarà:

  \forall (Valore, Dominio su cui itero, Condizione/Azione)

- **\exist** mi consente di dichiarare una condizione e verificare che esista **per almeno un valore**
- **La struttura** sarà:

  \exist (Valore, Dominio su cui itero, Condizione/Azione)

Keyword e funzioni utili : Negazione

- **\forall** mi consente di iterare lungo una serie di valori, come un «for»
- **La struttura** sarà:

  \forall (Valore, Dominio su cui itero, Condizione/Azione)

- **\exist** mi consente di dichiarare una condizione e verificare che esista **per almeno un valore**
- **La struttura** sarà:

  \exist (Valore, Dominio su cui itero, Condizione/Azione)

Entrambi possono essere negati, anteponendo un !
(stesse regole della logica del primo ordine)

Keyword e funzioni utili: \result

- **\result** consente di aggiungere condizioni e controlli sul risultato di un metodo

Keyword e funzioni utili : \old

- **\old** consente di far riferimento al vecchio stato di una variabile.

  Es: Se ho un metodo che aggiunge un elemento in una lista, in realtà dovrò specificare
  1. Che ho aggiunto l'elemento.
  2. Che non ho cambiato tutti gli altri

  Per poter dire che non ho cambiato tutti gli altri, potrò usare \old per riferirmi all'istanza della lista PRIMA della chiamata al metodo.

\forall int I, i >= 0 && i<list.size()-1, \old[i] == \result[i])

Keyword e funzioni utili : quantificatori

- **\max** consente di cercare il massimo data una certa condizione
- **\min** consente di cercare il minimo, data una certa condizione
- **\sum** consente di effettuare una sommatoria lungo un dominio
- **\product** consente di effettuare una «produttoria» lungo un dominio

Keyword e funzioni utili : num_of

- **\max** consente di cercare il massimo data una certa condizione
- **\min** consente di cercare il minimo, data una certa condizione
- **\sum** consente di effettuare una sommatoria lungo un dominio
- **\product** consente di effettuare una «produttoria» lungo un dominio

- **\num_of** consente di «contare» il numero di istanze che rispettano una certa condizione

Esercizio 1:

- public static boolean sottoStringa (char[] testo, char[] parola);
- ritorna true se e solo se parola è una sottostringa di testo

Esercizio 1:

- public static boolean sottoStringa (char[] testo, char[] parola);
- ritorna true se e solo se parola è una sottostringa di testo.

Hint:
ho degli oggetti, quindi il primo controllo da fare è che non siano null
inoltre, se una parola è inclusa nel testo…di cosa sono certo?

Esercizio 1: Soluzione – step 1

- public static boolean sottoStringa (char[] testo, char[] parola);
- ritorna true se e solo se parola è una sottostringa di testo

@requires

Esercizio 1: Soluzione – step 1

- public static boolean sottoStringa (char[] testo, char[] parola);
- ritorna true se e solo se parola è una sottostringa di testo

@requires testo != null && parola != null && testo.length >= parola.length

Dal momento che ho due oggetti, la prima cosa da fare è vedere che siano diversi da null.
Inoltre, se una parola deve essere inclusa nel testo, allora il testo deve essere più lungo della parola

Esercizio 1: Soluzione – step 1

- public static boolean sottoStringa (char[] testo, char[] parola);
- ritorna true se e solo se parola è una sottostringa di testo

@requires testo != null && parola != null && testo.length >= parola.length

Dal momento che ho due oggetti, la prima cosa da fare è vedere che siano diversi da null.
Inoltre, se una parola deve essere inclusa nel testo, allora il testo deve essere più lungo della parola

Osservando la funziona, cosa devo assegnare?

Esercizio 1: Soluzione – step 2

- public static boolean sottoStringa (char[] testo, char[] parola);
- ritorna true se e solo se parola è una sottostringa di testo

@requires testo != null && parola != null && testo.length >= parola.length

@assignable

Esercizio 1: Soluzione – step 2

- public static boolean sottoStringa (char[] testo, char[] parola);
- ritorna true se e solo se parola è una sottostringa di testo

@requires testo != null && parola != null && testo.length >= parola.length

@assignable \nothing

Dal momento che il metodo non modifica alcun oggetto o variabile, non c'è nulla da dichiarare assignable

Esercizio 1: Soluzione – step 2

- public static boolean sottoStringa (char[] testo, char[] parola);
- ritorna true se e solo se parola è una sottostringa di testo

@requires testo != null && parola != null && testo.length >= parola.length

Per quanto riguarda l'ensures invece… semplicemente, scorrendo gli array, ad un certo punto avrò elementi «uguali». In altre parole, esisteranno due indici tale che, da «quell'indice in poi», i caratteri di «parola» e «testo» coincidono

@assignable \nothing

Esercizio 1: Soluzione – step 3

- public static boolean sottoStringa (char[] testo, char[] parola);
- ritorna true se e solo se parola è una sottostringa di testo

@requires testo != null && parola != null && testo.length >= parola.length

@ensures

@assignable \nothing





Indice che rende vera l'ensures

Quando ci sono array, gli indici sono essenziali. **Parola** è contenuta in **testo** se, ad un certo punto, esisterà un indice di testo tale che, da quel punto in poi, i caratteri di testo e parola coincidono

Esercizio 1: Soluzione – step 3

- public static boolean sottoStringa (char[] testo, char[] parola);
- ritorna true se e solo se parola è una sottostringa di testo

@requires testo != null && parola != null && testo.length >= parola.length

@ensures (\exists int i; 0 <= i && i <= testo.length - parola.length -1;

@assignable \nothing

Quindi esiste un indice di «testo» tale che,





Indice che rende vera l'ensures

Esercizio 1: Soluzione – step 3

- public static boolean sottoStringa (char[] testo, char[] parola);
- ritorna true se e solo se parola è una sottostringa di testo

```
@requires testo != null && parola != null && testo.length >= parola.length
@ensures (\exists int i; 0 <= i && i <= testo.length - parola.length -1;
                    (\forall int j; 0 <= j && j <= parola.length-1;



@assignable \nothing
```

Quindi esiste un indice di «testo» tale che, per ogni indice di «parola»





Indice che rende vera l'ensures

Esercizio 1: Soluzione – step 3

- public static boolean sottoStringa (char[] testo, char[] parola);
- ritorna true se e solo se parola è una sottostringa di testo

@requires testo != null && parola != null && testo.length >= parola.length

@ensures (\exists int i; 0 <= i && i <= testo.length - parola.length -1;

(\forall int j; 0 <= j && j <= parola.length-1; testo[i+j] == parola[j])

@assignable \nothing





Indice che rende vera l'ensures

Quindi esiste un indice di «testo» tale che, per ogni indice di «parola», «testo» e «parola» coincidono

Esercizio 1: Soluzione – step 3

- public static boolean sottoStringa (char[] testo, char[] parola);
- ritorna true se e solo se parola è una sottostringa di testo

```
@requires testo != null && parola != null && testo.length >= parola.length
@ensures (\exists int i; 0 <= i && i <= testo.length - parola.length -1;
                    (\forall int j; 0 <= j && j <= parola.length-1; testo[i+j] ==
                        parola[j])
                ) ? \result == true : \result == false
@assignable \nothing
```





Indice che rende vera l'ensures

Quindi esiste un indice di «testo» tale che, per ogni indice di «parola», «testo» e «parola» coincidono. Se ciò è vero, il risultato è true, altrimenti è false

Esercizio 1: Soluzione – step 3

- public static boolean sottoStringa (char[] testo, char[] parola);
- ritorna true se e solo se parola è una sottostringa di testo

```
@requires testo != null && parola != null && testo.length >= parola.length
@ensures (\exists int i; 0 <= i && i <= testo.length - parola.length -1;
                        (\forall int j; 0 <= j && j <= parola.length-1; testo[i+j] ==
                                parola[j])
                        ) ? \result == true : \result == false
@assignable \nothing
```

Esercizio 2:

- public static void highLowNums (int [] nums, int [] highs, int n);

- l’array nums contiene interi tutti diversi tra di loro
- l’array highs è lungo esattamente n
- il metodo trova gli n numeri interi più grandi di nums e li inserisce in ordine decrescente nell’array highs
- l’array nums non viene modificato

Esercizio 2: Soluzione – step 1

@requires

Esercizio 2: Soluzione – step 1

@requires

Nums contiene tutti numeri diversi tra loro

Esercizio 2: Soluzione – step 1

```
@requires (\forall int i; 0 <= i && i <= nums.length-1;
                    (\forall int j; 0 <= j && j <= nums.length-1; (nums[i]
                        != nums[j] || i == j))) &&
```

Nums contiene tutti numeri diversi tra loro

quindi dati due indici, i e j, di nums, avremo che o i due indici sono uguali, oppure puntano sicuramente ad elementi diversi.

Esercizio 2: Soluzione – step 1

```
@requires (\forall int i; 0 <= i && i <= nums.length-1;
                    (\forall int j; 0 <= j && j <= nums.length-1; (nums[i]
                        != nums[j] || i == j))) &&
          (highs.length == n)
```

Nums contiene tutti numeri diversi tra loro

quindi dati due indici, i e j, di nums, avremo che o i due indici sono uguali, oppure puntano sicuramente ad elementi diversi.

Inoltre, sappiamo che la dimensione di highs è esattamente n

Esercizio 2: Soluzione – step 1

```
@requires (\forall int i; 0 <= i && i <= nums.length-1;
                    (\forall int j; 0 <= j && j <= nums.length-1; (nums[i]
                        != nums[j] || i == j))) &&
                (highs.length == n) && nums != null && highs != null
```

Nums contiene tutti numeri diversi tra loro

quindi dati due indici, i e j, di nums, avremo che o i due indici sono uguali, oppure puntano sicuramente ad elementi diversi.

Inoltre, sappiamo che la dimensione di highs è esattamente n

Inoltre sono oggetti, quindi entrambi **devono** essere diversi da null

Esercizio 2: Soluzione – step 2

```
@requires (\forall int i; 0 <= i && i <= nums.length-1;
                    (\forall int j; 0 <= j && j <= nums.length-1; (nums[i]
                        != nums[j] || i == j))) &&
                (highs.length == n) && nums != null && highs != null
```

Cosa devo assegnare?

Esercizio 2: Soluzione – step 2

```
@requires (\forall int i; 0 <= i && i <= nums.length-1;
                    (\forall int j; 0 <= j && j <= nums.length-1; (nums[i]
                        != nums[j] || i == j))) &&
          (highs.length == n) && nums != null && highs != null
```

L'array highs deve essere riempito con gli **n** numeri più grandi presenti in nums, quindi l'array highs verrà modificato

```
@assignable highs[*]
```

Esercizio 2: Soluzione – step 3

```
@requires (\forall int i; 0 <= i && i <= nums.length-1;
                    (\forall int j; 0 <= j && j <= nums.length-1; (nums[i]
                        != nums[j] || i == j))) &&
          (highs.length == n) && nums != null && highs != null

@ensures (\forall int i; 0 <= i && i <= nums.length -1; (
                    (\num_of int j; 0 <= j && j <= n - 1;
              nums[i]==highs[j]) <= 1)

@assignable highs[*]
```

esiste esattamente un elemento di nums[] per ogni elemento di highs []

Esercizio 2: Soluzione – step 3

```
@requires (\forall int i; 0 <= i && i <= nums.length-1;
                    (\forall int j; 0 <= j && j <= nums.length-1; (nums[i]
                        != nums[j] || i == j))) &&
          (highs.length == n) && nums != null && highs != null

@ensures (\forall int i; 0 <= i && i <= nums.length -1; (
                    (\num_of int j; 0 <= j && j <= n - 1;
          nums[i]==highs[j]) <= 1) &&
                    (\num_of int j; 0 <= j && j <= nums.length-1; highs[i]
                        < nums[j]) <= n-1)

@assignable highs[*]
```

Per ogni elemento di highs, ce ne sono al più n-1 in nums[] che sono più grandi di lui.
Se così non fosse, l'elemento di highs non sarebbe nei top **n**

Esercizio 2: Soluzione – step 3

```
@requires (\forall int i; 0 <= i && i <= nums.length-1;
                    (\forall int j; 0 <= j && j <= nums.length-1; (nums[i]
                        != nums[j] || i == j))) &&
          (highs.length == n) && nums != null && highs != null

@ensures (\forall int i; 0 <= i && i <= nums.length -1; (
                    (\num_of int j; 0 <= j && j <= n - 1;
          nums[i]==highs[j]) <= 1) &&
                    (\num_of int j; 0 <= j && j <= nums.length-1; highs[i]
                        < nums[j]) <= n-1) &&
                    (i == 0 || highs[i] < highs[i-1]))

@assignable highs[*]
```

Highs è ordinato in modo decrescente

Esercizio 2: Soluzione

```
@requires (\forall int i; 0 <= i && i <= nums.length-1;
                        (\forall int j; 0 <= j && j <= nums.length-1; (nums[i]
                                != nums[j] || i == j))) &&
                (highs.length == n) && nums != null && highs != null


@ensures (\forall int i; 0 <= i && i <= nums.length -1; (
                        (\num_of int j; 0 <= j && j <= n - 1;
                    nums[i]==highs[j]) <= 1) &&
                        (\num_of int j; 0 <= j && j <= nums.length-1; highs[i]
                                < nums[j]) <= n-1) &&
                        (i == 0 || highs[i] < highs[i-1]))

@assignable highs[*]
```

Esercizio 3

- public static boolean risultato(int[] x, int[] y);

true se x e' una permutazione di y, false altrimenti, sotto l'ipotesi che ne' x ne' y abbiano elementi duplicati.

Ovvero:
Controllare che x[] ed y[] contengano esattamente gli stessi elementi, anche se in ordine sparso

Esercizio 3: Soluzione – step 1

@requires

Esercizio 3: Soluzione – step 1

@requires x != null && y != null

Per prima cosa, gli oggetti non devono essere null

Esercizio 3: Soluzione – step 1

```
@requires x != null && y != null &&
(\forall int i; 0 <= i && i <= x.length-1;
        (\forall int j; 0 <= j && j <= x.length-1; (x[i] != x[j]) || i==j))
&&  (\forall int i; 0 <= i && i <= y.length-1; (\forall int j; 0 <= j && j <=
y.length-1; (y[i] != y[j]) || i == j))
```

Per prima cosa, gli oggetti non devono essere null
Inoltre, i due array non devono contenere duplicati

Esercizio 3: Soluzione – step 1

```
@requires x != null && y != null &&
(\forall int i; 0 <= i && i <= x.length-1;
        (\forall int j; 0 <= j && j <= x.length-1; (x[i] != x[j]) || i==j))
&&  (\forall int i; 0 <= i && i <= y.length-1; (\forall int j; 0 <= j && j <=
y.length-1; (y[i] != y[j]) || i == j))
```

Cosa devo assegnare?

Esercizio 3: Soluzione – step 2

@requires x != null && y != null &&
(\forall int i; 0 <= i && i <= x.length-1;
(\forall int j; 0 <= j && j <= x.length-1; (x[i] != x[j]) || i==j))
&& (\forall int i; 0 <= i && i <= y.length-1; (\forall int j; 0 <= j && j <= y.length-1; (y[i] != y[j]) || i == j))

Cosa devo assegnare?

@assignable \nothing

Esercizio 3: Soluzione – step 3

@requires x != null && y != null &&
(\forall int i; 0 <= i && i <= x.length-1;
(\forall int j; 0 <= j && j <= x.length-1; (x[i] != x[j]) || i==j))
&& (\forall int i; 0 <= i && i <= y.length-1; (\forall int j; 0 <= j && j <= y.length-1; (y[i] != y[j]) || i == j))

Cosa sappiamo delle post-condizioni?

@assignable \nothing

Esercizio 3: Soluzione – step 3

@requires x != null && y != null &&
(\forall int i; 0 <= i && i <= x.length-1;
(\forall int j; 0 <= j && j <= x.length-1; (x[i] != x[j]) || i==j))
&& (\forall int i; 0 <= i && i <= y.length-1; (\forall int j; 0 <= j && j <= y.length-1; (y[i] != y[j]) || i == j))

Cosa sappiamo delle post-condizioni?

1. Se Y è permutazione di X, allora la dimensione di X e Y devono essere uguali
2. Ogni elemento di X deve essere contenuto in qualche posizione di Y
3. Se entrambe queste cose sono vere, allora True, altrimenti False

@assignable \nothing

Esercizio 3: Soluzione – step 3

@requires x != null && y != null &&
(\forall int i; 0 <= i && i <= x.length-1;
(\forall int j; 0 <= j && j <= x.length-1; (x[i] != x[j]) || i==j))
&& (\forall int i; 0 <= i && i <= y.length-1; (\forall int j; 0 <= j && j <= y.length-1; (y[i] != y[j]) || i == j))

@ensures (x.length == y.length)

Cosa sappiamo delle post-condizioni?
1. Se Y è permutazione di X, allora la dimensione di X e Y devono essere uguali

@assignable \nothing

Esercizio 3: Soluzione – step 3

@requires x != null && y != null &&
(\forall int i; 0 <= i && i <= x.length-1;
(\forall int j; 0 <= j && j <= x.length-1; (x[i] != x[j]) || i==j))
&& (\forall int i; 0 <= i && i <= y.length-1; (\forall int j; 0 <= j && j <= y.length-1; (y[i] != y[j]) || i == j))

Cosa sappiamo delle post-condizioni?
2. Ogni elemento di X deve essere contenuto in qualche posizione di Y

@ensures (x.length == y.length)
&& (\forall int i; 0 <= i && i <= x.length-1;
(\exists int j; 0 <= j && j <= y.length-1; x[i] == y[j])) ?

@assignable \nothing

Esercizio 3: Soluzione – step 3

```
@requires x != null && y != null &&
(\forall int i; 0 <= i && i <= x.length-1;
        (\forall int j; 0 <= j && j <= x.length-1; (x[i] != x[j]) || i==j))
&&  (\forall int i; 0 <= i && i <= y.length-1; (\forall int j; 0 <= j && j <=
y.length-1; (y[i] != y[j]) || i == j))
```

Cosa sappiamo delle post-condizioni?

3. Se entrambe queste cose sono vere, allora True, altrimenti False

```
@ensures (x.length == y.length)
&& (\forall int i; 0 <= i && i <= x.length-1;
(\exists int j; 0 <= j && j <= y.length-1; x[i] == y[j])) ?
\result == true : \result == false

@assignable \nothing
```

Esercizio 3: Soluzione

@requires x != null && y != null &&
(\forall int i; 0 <= i && i <= x.length-1;
(\forall int j; 0 <= j && j <= x.length-1; (x[i] != x[j]) || i==j))
&& (\forall int i; 0 <= i && i <= y.length-1; (\forall int j; 0 <= j && j <= y.length-1; (y[i] != y[j]) || i == j))

@ensures (x.length == y.length)
&& (\forall int i; 0 <= i && i <= x.length-1;
(\exists int j; 0 <= j && j <= y.length-1; x[i] == y[j])) ?
\result == true : \result == false

@assignable \nothing

Esercizio 4:

- public static ArrayList<Integer> intervallo(int []x, int a, int b) throws Exception;

Dato un array di valori numerici interi (senza duplicati),
la funzione produce l'insieme dei valori che
ricadono all'interno di un intervallo chiuso a–b

Specificare che nel caso l'array contenga elementi duplicati **viene lanciata un'eccezione**.

Esercizio 4: Soluzione – step 1

@requires

Quali sono i pre-requisiti?

Esercizio 4: Soluzione – step 1

@requires x != null

Non viene specificato nulla nella traccia, ma abbiamo un oggetto, quindi un prerequisito è che sia diverso da null

Esercizio 4: Soluzione – step 1

@requires x != null

Non viene specificato nulla nella traccia, ma abbiamo un oggetto, quindi un prerequisito è che sia diverso da null

Cosa devo assegnare?

Esercizio 4: Soluzione – step 2

@requires x != null

@assignable \nothing

Il metodo non richiede di fare alcuna modifica

Esercizio 4: Soluzione – step 3

@requires x != null

@signals

@assignable \nothing

Esercizio 4: Soluzione – step 3

@requires x != null

@signals

@assignable \nothing

C'è un comportamento anomalo che va gestito con un'eccezione. Nello specifico, se ci sono duplicati, allora lancio l'eccezione.
ESISTONO ALTRI MODI PER DIRE «NON CI SONO DUPLICATI».
Dato che è un qualcosa di molto frequente…è bene ricordarne almeno un paio!

Esercizio 4: Soluzione – step 3

@requires x != null

@signals (Exception e) (\forall int i; 0 <= i && i <= x.length-1;(\num_of int j; 0 <= j && j <= x.length-1; x[i] == x[j]) > 1)

@assignable \nothing

C'è un comportamento anomalo che va gestito con un'eccezione. Nello specifico, se ci sono duplicati, allora lancio l'eccezione.

ESISTONO ALTRI MODI PER DIRE «NON CI SONO DUPLICATI».

Dato che è un qualcosa di molto frequente…è bene ricordarne almeno un paio!

Esercizio 4: Soluzione – step 4

@requires x != null

@ensures

@signals (Exception e) (\forall int i; 0 <= i && i <= x.length-1;(\num_of int j; 0 <= j && j <= x.length-1; x[i] == x[j]) > 1)

@assignable \nothing

Per ogni indice, diremo che l'elemento x[i] appartiene all'intervallo, **se e solo se** l'elemento è nel risultato e

Esercizio 4: Soluzione – step 4

@requires x != null

@ensures
(\forall int i; 0 <= i && i <= x.length-1;
    (x[i] >= a && x[i] <= b) <==>
    (\exists int j; 0 <= j && j <= \result.size()-1; x[i] == \result.get(j))) &&

@signals (Exception e) (\forall int i; 0 <= i && i <= x.length-1;(\num_of int j; 0 <= j && j <= x.length-1; x[i] == x[j]) > 1)

@assignable \nothing

Per ogni indice, diremo che l'elemento x[i] appartiene all'intervallo, **se e solo se** l'elemento è nel risultato e

Esercizio 4: Soluzione – step 4

@requires x != null

@ensures
(\forall int i; 0 <= i && i <= x.length-1;
    (x[i] >= a && x[i] <= b) <==>
    (\exists int j; 0 <= j && j <= \result.size()-1; x[i] == \result.get(j))) &&
(\forall int i; 0 <= i && i <= \result.size(); (
        (\result.get(i) >= a \result.get(i) <= b) &&
        (\exists int j; 0 <= j && j <= x.length-1; x[j] == \result.get(i))

@signals (Exception e) (\forall int i; 0 <= i && i <= x.length-1;(\num_of int j; 0 <= j && j <= x.length-1; x[i] == x[j]) > 1)

@assignable \nothing

Per ogni indice, diremo che l'elemento x[i] appartiene all'intervallo, **se e solo se** l'elemento è nel risultato e ogni elemento del risultato è compreso tra a e b **se e solo se** esiste un suo equivalente all'interno dell'array X

Esercizio 4: Soluzione – step 4

@requires x != null

@ensures
(\forall int i; 0 <= i && i <= x.length-1;
    (x[i] >= a && x[i] <= b) <==>
    (\exists int j; 0 <= j && j <= \result.size()-1; x[i] == \result.get(j))) &&
(\forall int i; 0 <= i && i <= \result.size(); (
        (\result.get(i) >= a \result.get(i) <= b) &&
        (\exists int j; 0 <= j && j <= x.length-1; x[j] == \result.get(i))

@signals (Exception e) (\forall int i; 0 <= i && i <= x.length-1;(\num_of int j; 0 <= j && j <= x.length-1; x[i] == x[j]) > 1)

@assignable \nothing

Esercizio 5:

- public static int[] substrings(char[] x,char[] y);

Si scrivano in JML le pre- e post- condizioni del seguente metodo Substrings.

Questo prende in ingresso due array di caratteri x e y e restituisce, in un array di interi, l'elenco, in ordine qualunque, di tutti e soli gli indici in cui y compare come sottostringa di x.

- Ad esempio, se x = [abbbbbabbbabbbabb] e y = [abb], il

metodo restituisce l'array [0 6 10 14].

Esercizio 5 – Soluzione

@requires

Qual è il pre-requisito?

Esercizio 5 – Soluzione

@requires x!=null && y!=null

Qual è il pre-requisito?

Esercizio 5 – Soluzione

@requires x!=null && y!=null

@ensures

Esercizio 5 – Soluzione

@requires x!=null && y!=null

@ensures \result!=null &&

Il risultato è diverso da null e

Esercizio 5 – Soluzione

@requires x!=null && y!=null

@ensures \result!=null && (x.length < y.length) ? \result.length==0

Il risultato è diverso da null e, se la lunghezza di x è minore di quella di y, allora il risultato avrà lunghezza 0, poiché non possono esserci sottostringhe. Se però Ciò non è vero, ovvero se la lunghezza di X è superiore a Y
allora

Esercizio 5 – Soluzione

@requires x!=null && y!=null

@ensures \result!=null && (x.length < y.length) ? \result.length==0 :

```
\forall(int k; ;
        ( \forall(int p; p>=0 && p<y.length; x[k+p]==y[p]) &&
                k>=0 && k<x.length - y.length) <==>
        \exists(int idx; idx>=0 && idx<result.length; result[idx]==k)
        )
)
```

Per ogni numero k, avremo che esiste una P tale che P è sottostringa di K, e K valido (ovvero compreso tra 0 e x.length – y.length se e solo se quel valore k è anche contenuto nel risultato

Esercizio 5 – Soluzione

```
@requires x!=null && y!=null
@ensures \result!=null && (x.length < y.length) ? \result.length==0 :
        \forall(int k; ;
                ( \forall(int p; p>=0 && p<y.length; x[k+p]==y[p]) &&
                        k>=0 && k<x.length - y.length) <==>
                \exists(int idx; idx>=0 && idx<result.length; result[idx]==k)
                )
        )
```

Esercizio 6:

- public static int confronta(int[] a, int[] b);

Si scriva la specifica JML del metodo imponendo che a e b non contengano elementi ripetuti.

Il metodo restituisce: 0 se i due array passati come parametri non contengono gli stessi elementi; 1 se contengono gli stessi elementi, ma in ordine diverso; 2 se contengono gli stessi elementi, nello stesso ordine.

Quali sono i prerequisiti?

Esercizio 6: Soluzione

\\@requires: a != null && b != null && a e b devono essere diversi da null

Esercizio 6: Soluzione

\\@requires: a != null && b != null &&

Non esistono duplicati

Esercizio 6: Soluzione

```
\\@requires: a != null && b != null &&
        \forall(int i; i >= 0 && i <= a.length ;
            !(\exist int j ; j >= 0 && j < a.length ; i != j && a[i] == a[j])
        ) &&
        \forall(int k; k >= 0 && k <= b.length ;
            !(\exist int q ; q >= 0 && q < b.length ; k != q && b[k] == b[q])
        )
```

Non esistono duplicati

Esercizio 6: Soluzione

```
\\@requires: a != null && b != null &&
        \forall(int i; i >= 0 && i <= a.length ;
            !(\exist int j ; j >= 0 && j < a.length ; i != j && a[i] == a[j])
        ) &&
        \forall(int k; k >= 0 && k <= b.length ;
            !(\exist int q ; q >= 0 && q < b.length ; k != q && b[k] == b[q])
        )
```

Cosa possiamo dire dell'ensures?

Esercizio 6: Soluzione

```
\\@requires: a != null && b != null &&
            \forall(int i; i >= 0 && i <= a.length ;
                  !(\exist int j ; j >= 0 && j < a.length ; i != j && a[i] == a[j])
             ) &&
             \forall(int k; k >= 0 && k <= b.length ;
                  !(\exist int q ; q >= 0 && q < b.length ; k != q && b[k] == b[q])
             )

\\@ensures: \result!= null &&
```

Il risultato è diverso da null e:

Esercizio 6: Soluzione

```
\\@requires: a != null && b != null &&
            \forall(int i; i >= 0 && i <= a.length ;
                !(\exist int j ; j >= 0 && j < a.length ; i != j && a[i] == a[j])
            ) &&
            \forall(int k; k >= 0 && k <= b.length ;
                !(\exist int q ; q >= 0 && q < b.length ; k != q && b[k] == b[q])
            )

\\@ensures: \result!= null && ((a.length != b.length) Il risultato è diverso da null e:
                                                      se le lunghezze sono diverse…
```

Esercizio 6: Soluzione

```
\\@requires: a != null && b != null &&
            \forall(int i; i >= 0 && i <= a.length ;
                  !(\exist int j ; j >= 0 && j < a.length ; i != j && a[i] == a[j])
             ) &&
             \forall(int k; k >= 0 && k <= b.length ;
                  !(\exist int q ; q >= 0 && q < b.length ; k != q && b[k] == b[q])
             )

\\@ensures: \result!= null && ((a.length != b.length) ||
            \exist( int i , i >= 0 && i < a.length ,
                  \forall(int j, j >= 0 && j < a.length, b[j] != a[i])
                   )
```

O sono diversi per almeno un carattere

Esercizio 6: Soluzione

```
\\@requires: a != null && b != null &&
            \forall(int i; i >= 0 && i <= a.length ;
                  !(\exist int j ; j >= 0 && j < a.length ; i != j && a[i] == a[j])
             ) &&
             \forall(int k; k >= 0 && k <= b.length ;
                  !(\exist int q ; q >= 0 && q < b.length ; k != q && b[k] == b[q])
             )

\\@ensures: \result!= null && ((a.length != b.length) ||
           \exist( int i , i >= 0 && i < a.length ,
                  \forall(int j, j >= 0 && j < a.length, b[j] != a[i])
                   )
           ) ? \result ==  0 :
```

E Il risultato deve essere 0.
Altrimenti….

Esercizio 6: Soluzione

```
\\@requires: a != null && b != null &&
            \forall(int i; i >= 0 && i <= a.length ;
                  !(\exist int j ; j >= 0 && j < a.length ; i != j && a[i] == a[j])
             ) &&
             \forall(int k; k >= 0 && k <= b.length ;
                  !(\exist int q ; q >= 0 && q < b.length ; k != q && b[k] == b[q])
             )

\\@ensures: \result!= null && ((a.length != b.length) ||
            \exist( int i , i >= 0 && i < a.length ,
                  \forall(int j, j >= 0 && j < a.length, b[j] != a[i])
                   )
            ) ? \result ==  0 :
            ( \forall(int k , k >= 0 && k <= a.length, a[k] == b[k]) ? \result == 2 :
                  \result == 1)
```

Se per ogni k, a e b sono uguali,
il risultato è 2. Altrimenti restituisco 1

Esercizio 6: Soluzione

```
\\@requires: a != null && b != null &&
            \forall(int i; i >= 0 && i <= a.length ;
                  !(\exist int j ; j >= 0 && j < a.length ; i != j && a[i] == a[j])
             ) &&
             \forall(int k; k >= 0 && k <= b.length ;
                  !(\exist int q ; q >= 0 && q < b.length ; k != q && b[k] == b[q])
             )

\\@ensures: \result!= null && ((a.length != b.length) ||
           \exist( int i , i >= 0 && i < a.length ,
                  \forall(int j, j >= 0 && j < a.length, b[j] != a[i])
                    )
           ) ? \result ==  0 :
           ( \forall(int k , k >= 0 && k <= a.length, a[k] == b[k]) ? \result == 2 :
                  \result == 1)
```

Esercizio 7:

- public static int conta(char[] c, int[] i);

Si scriva la specifica in JML del metodo.

- Il metodo prende in ingresso due array: c è un array di caratteri e i un array di interi.
- Il metodo controlla che per ogni elemento e in i ci siano esattamente e caratteri uguali in c e viceversa.

Se il controllo dà esito positivo, il metodo restituisce il totale dei caratteri letti; 0 in caso contrario.

- Ad esempio, se c contenesse: a, a, b, c, b, a; i dovrebbe
- contenere: 2, 1, 3 e il risultato sarebbe 6.

Esercizio 7: Soluzione

\\@requires c != null && i != null

\\@ensures \result >= 0 &&

\forall (int idx, idx >= 0 && idx < i.length, // Per ogni idx da 1 a length(i)

\exist(int j, j >= 0 && j < c.length, //deve esistere un indice j in c

//tale che il numero di caratteri uguali a c[j] (\num_of int k, k >=0 && k < i.length,
// è esattamente uguale ad i[idx] c[k] == c[j]) == i[idx]

) ? \result == \sum(int n, n >= 0 && n < i-length, i[n]) : \result == 0

Esercizio 7: Soluzione (più o meno)

```
\\@requires c != null && i != null


\\@ensures \result >= 0 &&
            \forall (int idx, idx >= 0 && idx < i.length, // Per ogni idx da 1 a length(i)
                \exist(int j, j >= 0 && j < c.length, //deve esistere un indice j in c
//tale che il numero di caratteri uguali a c[j] (\num_of int k, k >=0 && k < i.length,
// è esattamente uguale ad i[idx]                                    c[k] == c[j]) == i[idx]
        ) ? \result == \sum(int n, n >= 0 && n < i-length, i[n]) : \result == 0
```

Sembra corretto…eppure non lo è! Perché?

Esercizio 7:

- public static int conta(char[] c, int[] i);

Si scriva la specifica in JML del metodo.

- Il metodo prende in ingresso due array: c è un array di caratteri e i un array di interi.
- Il metodo controlla che per ogni elemento e in i ci siano esattamente e caratteri uguali in c e viceversa.

Se il controllo dà esito positivo, il metodo restituisce il totale dei caratteri letti; 0 in caso contrario.

- Ad esempio, se c contenesse: a, a, b, c, b, a; i dovrebbe
- contenere: 2, 1, 3 e il risultato sarebbe 6.

Questa condizione mi impone di controllare che, se l'array i ha due valori uguali, questi devono corrispondere a caratteri diversi.

Esercizio 7: Soluzione

```
@requires c != null && i != null
@ensures (\forall int j; 0 <= j && j <= i.length-1;
                (\num_of char k; 0 <= k && k <= 255;
                    (\num_of int l; 0 <= l && l <= c.length-1; c[l] == k) == i[j])
                == (\num_of int l; 0 <= l && l <= i.length-1; i[j] == i[l]) ?
                        \result == c.length : \result == 0
```

Occorre cambiare l'exist, in quanto non si può confrontare un exist con un num_of

Esercizio 8

- public static int rigaSommaMax(int[][] m);

Si scriva la specifica in JML del metodo.

- Il metodo prende in ingresso una matrice rettangolare con almeno 2 righe e 2 colonne.
- Il metodo ritorna l'indice della prima riga i cui elementi hanno somma massima.

Esercizio 8

- public static int rigaSommaMax(int[][] m);

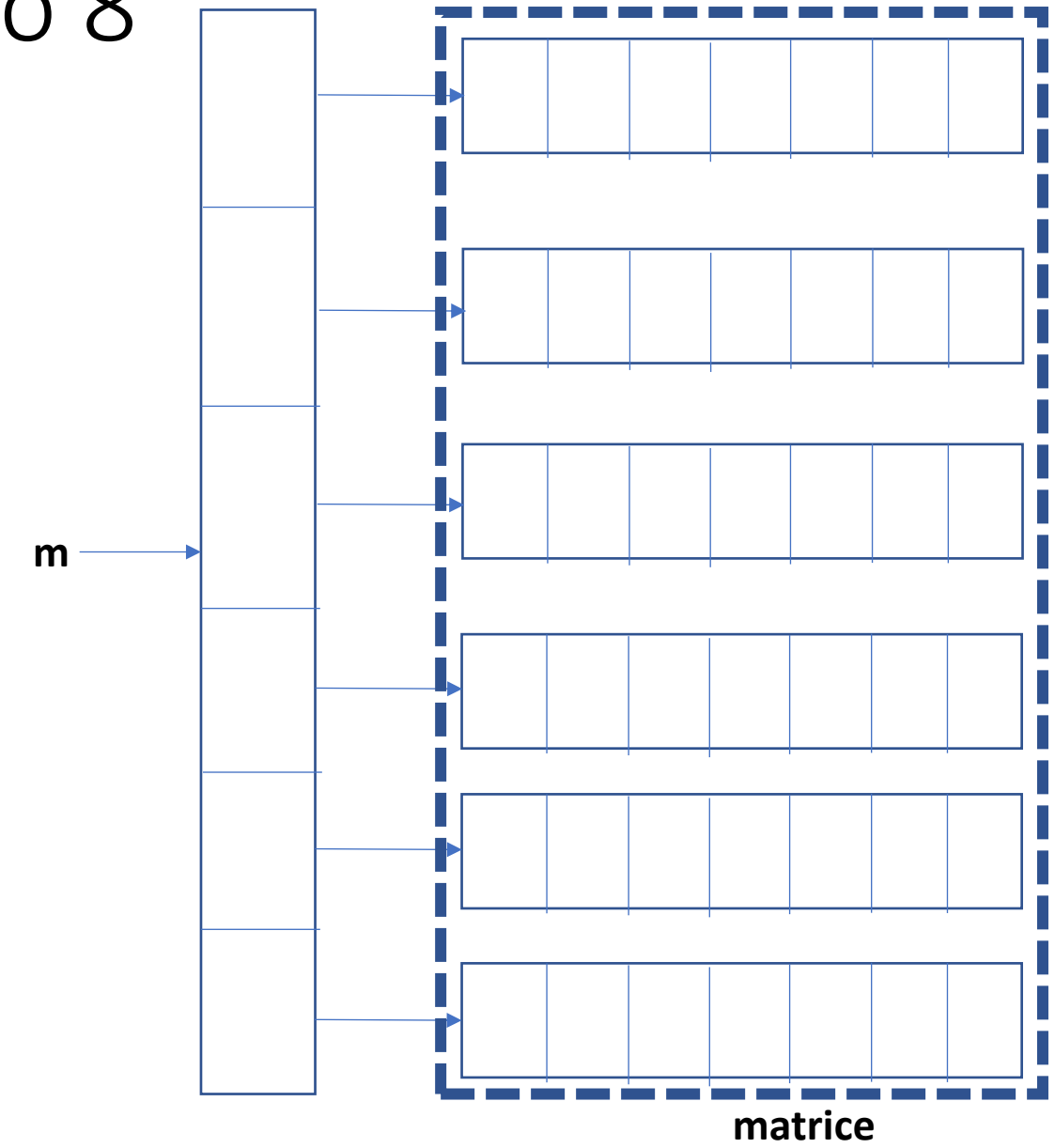
Si scriva la specifica in JML del metodo.

- Il metodo prende in ingresso una matrice rettangolare con almeno 2 righe e 2 colonne.
- Il metodo ritorna l'indice della prima riga i cui elementi hanno somma massima.

Hint: ricordate come sono fatti gli array in java. Una matrice è un array di array. Quindi prima controllo che la length di **m** sia maggiore di 2, poi controllo che la length di ogni elemento all'interno di **m** sia maggiore di 2

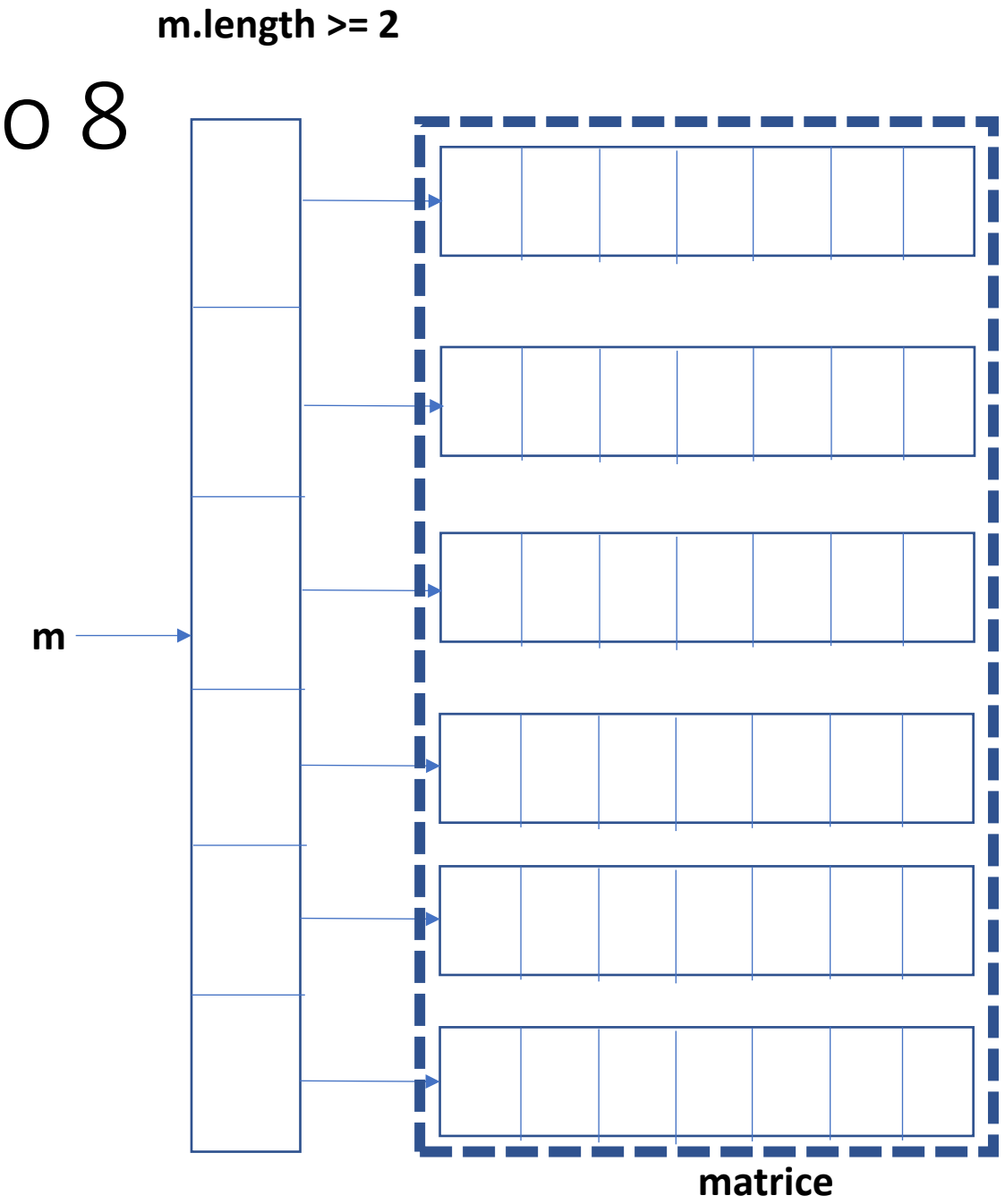
Esercizio 8

\\@requires



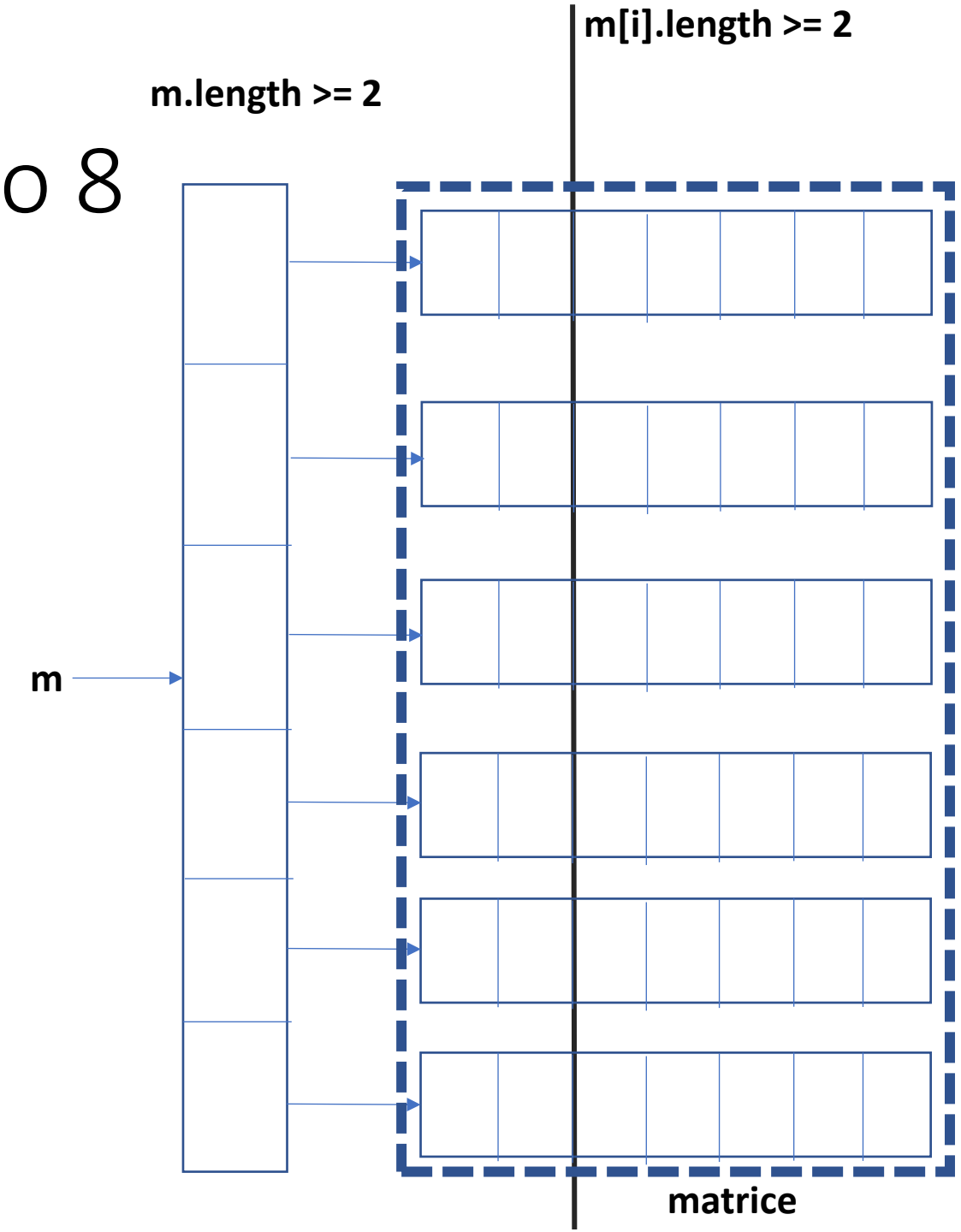
Esercizio 8

\\@requires



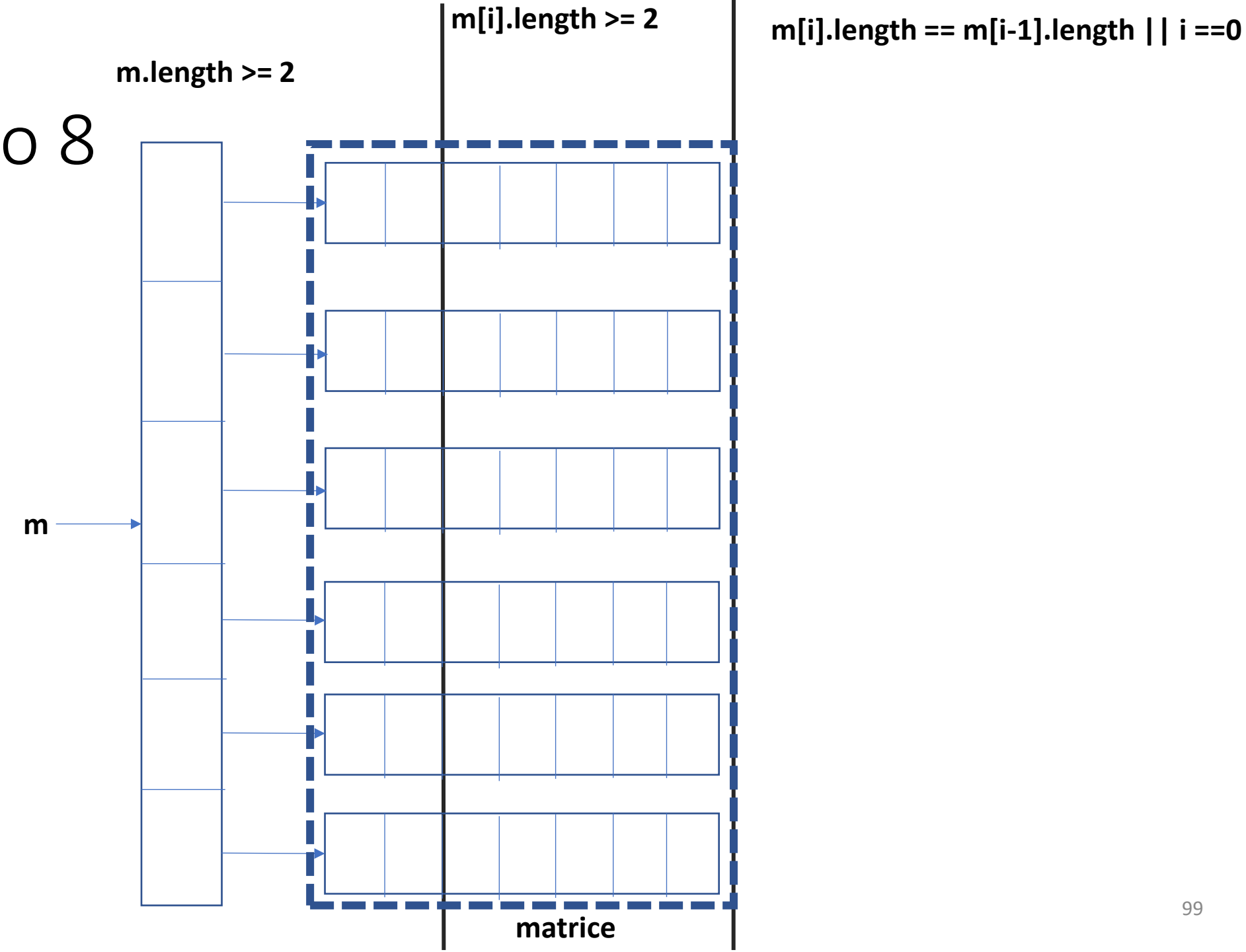
Esercizio 8

\\@requires



Esercizio 8

\\@requires



Esercizio 8: Soluzione

```
@requires m != null && (m.length >= 2 &&
                (\forall int i; 0 <= i && i <= m.length-1; m[i].length >= 2 &&
                                                (i == 0 ||m[i].length ==
                                                                m[i-1].length)))
@ensures

@assignable \nothing
```

Per prima cosa, il risultato deve essere una riga accettabile

Esercizio 8: Soluzione

```
@requires m != null && (m.length >= 2 &&
                (\forall int i; 0 <= i && i <= m.length-1; m[i].length >= 2 &&
                                                (i == 0 ||m[i].length ==
                                                                m[i-1].length)))
@ensures (0 <= \result && \result <= m.length-1)
 &&


@assignable \nothing
```

Per prima cosa, il risultato deve essere una riga accettabile

Esercizio 8: Soluzione

```
@requires m != null && (m.length >= 2 &&
                (\forall int i; 0 <= i && i <= m.length-1; m[i].length >= 2 &&
                                                (i == 0 ||m[i].length ==
                                                                m[i1].length)))
@ensures (0 <= \result && \result <= m.length-1)
 && (\forall int i; 0 <= i && i < \result;
        (\sum int j; 0 <= j && j <= m[i].length-1; m[i][j]) <
        (\sum int k; 0 <= k && k <= m[\result].length-1; m[\result][k])) &&
```

```
@assignable \nothing
```

Dopodichè, controllo che tutti gli elementi **PRIMA** dell'indice \result contengono TUTTE somme inferiori. Infatti \result deve essere LA PRIMA di quelle con somma maggiore

Esercizio 8: Soluzione

```
@requires m != null && (m.length >= 2 &&
                (\forall int i; 0 <= i && i <= m.length-1; m[i].length >= 2 &&
                                                (i == 0 ||m[i].length ==
                                                                m[i1].length)))
@ensures (0 <= \result && \result <= m.length-1)
 && (\forall int i; 0 <= i && i < \result;
        (\sum int j; 0 <= j && j <= m[i].length-1; m[i][j]) <
        (\sum int k; 0 <= k && k <= m[\result].length-1; m[\result][k])) &&
!(\exists int i; \result+1 <= i && i <= m.length-1;
        ((\sum int j; 0 <= j && j <= m[i].length-1; m[i][j]) >
        (\sum int k; 0 <= k && k <= m[\result].length-1; m[\result][k]))
@assignable \nothing
```

In ultimo, dato che \result è la **prima** con somma maggiore, mi interessa controllare che non esistano, indici successivi a \result, che diano una somma maggiore di quella su \result.

Esercizio 8: Soluzione

```
@requires m != null && (m.length >= 2 &&
                (\forall int i; 0 <= i && i <= m.length-1; m[i].length >= 2 &&
                                                (i == 0 ||m[i].length ==
                                                                m[i1].length)))
@ensures (0 <= \result && \result <= m.length-1)
 && (\forall int i; 0 <= i && i < \result;
        (\sum int j; 0 <= j && j <= m[i].length-1; m[i][j]) <
        (\sum int k; 0 <= k && k <= m[\result].length-1; m[\result][k])) &&
!(\exists int i; \result+1 <= i && i <= m.length-1;
        ((\sum int j; 0 <= j && j <= m[i].length-1; m[i][j]) >
        (\sum int k; 0 <= k && k <= m[\result].length-1; m[\result][k]))
@assignable \nothing
```